Домашнее задание 1. Git, github, CI/CD.

Формат сдачи:

Все домашние задания по курсу сдаются как страничка в Github-репозитории, который структурирован так, как указано в конце задания ниже. Пожалуйста, обратите внимание на взаимное расположение файлов в репозитории и на их название. Я буду запускать автоматический базовый чекер, который проверяет наличие файлов в репозитории для каждой домашки. Для каждого задания старайтесь хранить листинг команд, которыми вы пользовались – они понадобятся для сдачи. Часто ответы на задание нужно будет писать в README.md в какой-то папке репозитория (воспринимайте это как что-то похожее на домашки по практической биоинформатике).

Уточнение:

Если у вас не Linux, вы можете воспользоваться либо WSL2 (устанавливается автоматически в Windows > 10), либо установить cygwin: **cygwin.org**, позволяющий работать в командной строке. По всем вопросам пишите в чат курса или в личные сообщения преподавателям. При работе с git не забывайте как можно чаще коммитить изменения. В markdown можно вставлять картинки и таблицы – не стесняйтесь это делать, если чувствуете, что это хорошо иллюстрирует ситуацию.

Структура репозитория после первого ДЗ:

```
.
├── .github
│   └── workflows
│       └── HW1-WF.yaml
├── .gitignore
├── hw1
│   ├── complement.py
│   ├── count_kmers.py
│   ├── lgb.sh
│   ├── README.md
│   └── test.fna
├── hw2
│   └── ...
├── hw3
│   └── ...
├── hw4
│   └── ...
├── hw5
│   └── ...
└── README.md
```

Задание 1. Репозиторий.

Это задание нужно для того, чтобы после курса у вас точно остался аккаунт на github, если вдруг его раньше не было.

1. Создайте аккаунт на Github, если ещё не создали на занятии.

   Как и на занятии, пробросьте ssh-ключ в репозиторий. Проверьте, что у вас получается зайти на Github с командной строки:

   `Hi <user>! You've successfully authenticated, but GitHub does not provide shell access.`

2. Создайте репозиторий для домашних заданий, **сделайте его приватным**. Добавьте меня (**`dkhlebn`**) в коллабораторы.

   При создании репозитория добавьте **`README.md`** сразу, можете добавить **`.gitignore`** для своего удобства. Приватность репозитория задаётся при его создании. Назвать репозиторий можете как угодно. Добавить коллаборатора можно в репозитории на вкладке **Settings -> Collaborators -> Add People**.

Задание 2. Работа с git.

Это задание для ознакомления с системой и командами git на базовом уровне.

1. Склонируйте свой репозиторий для домашек себе на ПК / ноутбук.
2. Напишите Python-код в файл **`hw1/complement.py`**.

   Код принимает на вход с командной строки нуклеотидную последовательность в и печатает её reverse-complement и GC-состав до 3 знака после запятой. Сделайте так, чтобы программу можно было запускать без передачи интерпретатора в CLI:

   ```
   ~$ ./complement.py --seq ATGCCGATGG
   CCAGCGGCAT
   0.600
   ```

3. Отправьте этот файл в репозиторий (с помощью коммитов и пуша).

   Вы можете отправлять и несколько коммитов с последовательным изменением и добавлением фичей в код, чтобы была более богатая история коммитов.

4. Если интересно, можете положить в эту папку ещё что-то, поисследовав **`git add / rm / mv`**.
5. Присвойте последнему HEAD-коммиту тег **`task1`**.
6. В **`README.md`** папки **`hw1`** добавьте описание всех файлов.

   Разберитесь с синтаксисом github markdown (он мало отличается от языка разметки в Jupyter-ноутбуках и Google Colab). Сделайте красивое оформление с помощью markdown.

Задание 3. Работа с конфликтами.

Здесь мы попробуем имитацию конфликта изменений между пользователями одного репозитория. Такое на практике у вас будет происходить очень часто.

1. Напишите Python-код в файл **`hw1/count_kmers.py`**.

   Код принимает на вход с командной строки имя **`fasta`**-файла и выводит в выходной файл **`cnts.json`** количество 4-меров в последовательностях **`fasta`**-файла.

   ```
   ~$ ./count_kmers.py --fa test.fna
   ~$ cat cnts.json
   {
     “seq1”: {“ATAT”:3, “ATGC”: 1},
     “seq2”: {“WKRF”:1, “KRFP”: 1}
   }
   ```

2. Отправьте файл в репозиторий (с помощью коммитов и пуша).
3. Отредактируйте этот код на github.

   Зайдите на страницу кода на Github и внесите какие-нибудь изменения в этот код. Например, замените значение k с 4 на 2. Так мы имитируем то, что вы работаете в команде, и это значение, скажем, внёс в репозиторий ваш коллега.

4. Допишите код **`hw1/count_kmers.py`**.

   Внесите в локальном слепке репозитория на своём ПК изменения в код. Поменяйте структуру кода так, что теперь он принимает на вход не только название входного файла, но название выходного файла и значение **`k`**.

   ```
   ~$ ./count_kmers.py --fa test.fna -k 3 --out out.json
   ~$ cat out.json
   {
     “seq1”: {“ATA”: 3, “TAT”: 3, “ATG”: 1, “TGC”: 1},
     “seq2”: {“WKR”: 1, “KRF”: 1, “RFP”: 1}
   }
   ```

5. Попытайтесь закоммитить изменения.

   Что происходит? Почему?

6. Разрешите конфликт на стадии push.

   Решите конфликт – определите, какие изменения и как остаются. Так как вы изменяли файл в обоих случаях, договариваться с собой проще.

7. Опишите все действия, события и принятые решения в файле **`hw1/README.md`**.

Задание 4. Создание PR.

Это задание обязательно для выполнения вне зависимости от того, как вы планируете получать зачёт по курсу. Считайте, что это ваша “регистрация на курс”.

1. Сделайте себе форк моего репозитория на локальное устройство.

   Адрес репозитория: **github.com/dkhlebn/FBB-orchestration**

2. В вашем форке сделайте новую ветку.

   Сделайте ветку **`username_accept`** где **`username`** – ваш никнейм на Github. Зайдите в эту ветку.

3. В папке **`accepts_2025`** создайте файл согласия с правилами курса и тем, как вы будете его сдавать.

   Не переживайте о решении сейчас: делать пулл-реквест в репозиторий можно делать неограниченное количество раз, вы сможете изменить решение о вашем формате сдачи курса позже. Вы должны сделать YAML-файл **`username.yaml`** (**`username`** – ваш никнейм на github) следующего формата:

```
github_username: <your_username>

first_name: <Your Name>
last_name: <Your Last Name>
repo: <URL or None>
grading: <homeworks OR project>

agreement: |
  Я подтверждаю следующее:
  1. Я не буду списывать решения у других участников курса.
  2. При использовании больших языковых моделей (LLM) или
  других генеративных ИИ-инструментов для выполнения домашних
  заданий я обязуюсь полностью понимать присланный код и быть
  готовым пояснить, как он работает, почему используется та
  или иная конструкция, а также внести в него изменения по
  запросу преподавателя.

  Нарушение этих правил может повлечь за собой исключение
  из курса или аннулирование результатов.

agree_to_rules: yes
```

   Скопировать текст-форму в файл вы можете из текста **`README.md`** в директории **`accepts_2025`**. Если планируете сдавать зачет проектом, в поле **`repo`** укажите **`None`**, иначе адрес репозитория.

4. Сделайте коммит ваших изменений в свой локальный репозиторий.

5. Попробуйте сделать push.

   Скорее всего, это сделать не выйдет – ваша ветка существует локально, но её нет ни в вашем форке, ни тем более в исходном репозитории. Для правильной работы с пушем ветки вам нужно будет её одновременно запушить и в ваш форк:

   `git push -u origin username_accept`

6. Создайте pull-request с помощью UI Github.
   Обратите внимание! Заголовок PR должен быть отформатирован так: `acceptance-orch2025-username` (`username` – ваш никнейм на github). Это – название вашего PR, если он будет отформатирован не так, его автоматически не выйдет принять.

Задание 5. Практика работы с ветками (доп.).

Хорошо, что в сети существуют разные ресурсы для тренировки любых навыков. Один из них: **learngitbranching.js.org**.

1. Пройдите туториалы на указанном ресурсе, сохраняйте команды себе в листинг.
2. В файл `hw1/lgb.sh` сохраните листинг команд, позволяющий решить задачи на этом ресурсе. Отделяйте задания строкой комментария.
3. Добавьте описание созданного файла в `hw1/README.md`.

Задание 6. Автоматизация через Github Actions.

В этой задаче от вас потребуется создать workflow, проверяющий работоспособность скриптов, созданных в задании 2 и 3. Не забывайте описывать все ваши действия в `README`.

1. Создайте тестовый `fasta`-файл в `hw1`.
   Положите туда где-то 5 последовательностей, как нуклеотидных, так и белковых.
2. В `.github/workflows` положите файл с тестированием вашего кода на push в репозитории.
   Вам понадобится `actions@checkout@v4`, `actions@setup-python@v5`. Напишите воркфлоу, который делает чекаут вашего репозитория, устанавливает питон, а затем проверяет, что скрипты из задания 2 и 3 не падают. Проверку скрипта на обратную комплементарную последовательность можете осуществлять с помощью любой последовательности, проверку подсчёта k-меров – с помощью `fasta`-файла из предыдущего пункта.
3. Защитите ветку main от непрошедших тесты слияний.
   Это можно сделать в **Settings** -> **Branches** -> **Branch protection rules.** Установите галочку в пункте **"Require status checks to pass before merging".** Выберите ваш воркфлоу, который получили на прошлом шаге.
4. Создайте новую ветку `workflow-testing` и сломайте один из скриптов.
   В новой ветке сделайте так, чтобы скрипт выдавал ошибку. Сделайте коммит изменений и попытайтесь сделать merge с таргетной веткой `main`. Опишите наблюдения, что выдал Github Actions. Почините любым способом изменения и добавьте в питоновские скрипты

логирование действий (глубина логирования – по желанию). Сделайте коммиты изменений и попытайтесь сделать `merge`. Посмотрите, что будет в этом случае. Опишите наблюдения и чувства от увиденной галочки о том, что воркфлоу прошёл.